Institute of Technology of Cambodia

Department of Information and Communication Engineering

Distributed Systems

TP-03

Group: I4-GIC-B

Lecturer: KHEM Thay **(Cours & TP)**

Student Name	ID
PHOUNG Sovannara	e20230093
PISETH Panhavorn	e20230892
SOK Vireak	e20231270
SUN Sovisal	e20230105
THA Winner	e20230181

Academic Year 2026 – 2027

Chapter 2: System model

1. Explain the concept of **remote desktop**, and give EX of TeamViewer or VNC or MS

Remote Desktop.

2. Explain **semaphores** in OS and DB.

3. Explain 1 of the topics below:

- The emergence of **mobile computing**
- The emergence of **ubiquitous computing**
- The emergence of **cloud computing**

4. Explain the process how **torrent** works.

5. Find the advantages of peer-to-peer over client-server architecture.

6. Between **client-server** and **peer-to-peer**, which one is more **complex**, explain.

7. Are **multiple servers** and **cluster** the same? Explain.

8. Explain proxy server.

1. Concept of remote desktop

Remote desktop is a technology that allows a user to see and control another computer through a network (LAN or Internet), as if sitting in front of it. The programs run on the remote computer. Only the **screen image** is sent to the user, and only the **keyboard and mouse actions** are sent back. It follows the **client-server model**: the user's device is the client, and the remote computer is the server (host).

How it works:

- The client connects to the remote computer.
- The user enters a password to get access.
- The remote computer sends its screen to the client.
- The client sends mouse and keyboard actions, and the remote computer executes them.

Example: TeamViewer

TeamViewer is a popular remote desktop software that works on Windows, macOS, Linux, Android, and iOS.

- Both computers open TeamViewer. Each one gets a unique **ID** and **password**.
- The user enters the partner's ID and password.
- TeamViewer's servers find the partner and create the connection, even through firewalls.
- The user sees the partner's screen and controls it. The connection is protected with **end-to-end encryption**.

For example, an IT technician can fix a customer's computer from another city without traveling.

2. Semaphores: OS vs. DB

A **semaphore** is an integer counter controlled by two atomic operations: wait() (decrements; blocks if zero) and signal() (increments; unblocks waiting threads).

- In Operating Systems (OS)

Role: Kernel-level CPU, process, and thread synchronization.
Key Uses:
Critical Sections: Prevents race conditions on shared memory and hardware devices.
Process Signaling: Coordinates execution order (e.g., Producer-Consumer queues).
Implementation: Blocks threads via Task Control Blocks (TCBs), placing them in sleep queues to avoid busy-waiting.

- In Database Systems (DBMS)

Role: Low-level engine memory protection and worker throttling (not ACID user locks).

Key Uses:
Connection Pools: Limits concurrent active queries to available slots.
Buffer Pool Frames: Coordinates access to shared memory pages during disk reads/writes.
WAL Flushing: Signals background log-writer threads to fsync transactions to disk.

Engine Latches: Backs lightweight in-memory data structure locks (B-trees, hash buckets) when threads must sleep.

3. Explain 1 of the topics below

- The Emergence of Mobile Computing: A trend in contemporary distributed systems where hardware devices (laptops, smartphones, PDAs) are portable and location-independent. Users can maintain network connectivity via wireless technologies (Wi-Fi, 3G/4G/5G, GPS) while physically moving across different host networks, enabling location-aware applications and services.
- The Emergence of Ubiquitous Computing: The harnessing of numerous small, inexpensive computational devices embedded directly into users' physical environments (homes, offices, or natural settings). These embedded devices communicate seamlessly in the background to assist users invisibly without requiring conscious interaction.
- The Emergence of Cloud Computing: The delivery of computing infrastructure, storage, and software platforms as a scalable, pay-as-you-go utility over the Internet. Cloud systems use virtualization to abstract physical hardware into elastic resource pools that automatically scale up or down based on workload demand.

4. Explain the process of how torrent works.

BitTorrent is a **peer-to-peer (P2P)** file-sharing protocol. Instead of downloading a file from one central server, a user downloads **small pieces** of the file **from many other users at the same time**, and also **uploads** pieces to others. So each user is both a **client and a server**.

Main terms:

- **Seeder:** a user who has the complete file and shares it.
- **Leecher:** a user who is still downloading (and also shares the pieces they already have).
- **Swarm:** all users sharing the same file.
- **Tracker:** a server that keeps the list of users in the swarm. Peers can also be found without a tracker using **DHT** (Distributed Hash Table).

How it works:

1. The file is split into **small pieces**, and a **.torrent file** or **magnet link** is created. It contains the file information and a **hash** for each piece.

2. The user opens the torrent in a client such as **qBittorrent** or **uTorrent**.
3. The client contacts the **tracker** or **DHT** to find other peers who have the file.
4. The client downloads **different pieces from different peers at the same time**, usually the **rarest pieces first**.
5. Each piece is checked with its **hash**. Broken or fake pieces are downloaded again.
6. While downloading, the client also **uploads** its pieces to other peers.
7. When all pieces are complete, they are joined into the full file, and the user can continue as a **seeder**.

Example: a 1 GB file is split into 4,000 pieces. You get some pieces from a user in Cambodia, some from Japan, and some from France, all at the same time, while you send your pieces to others.

Advantages: fast download, **scalable** (more users means more uploaders), and **no single point of failure**.

Disadvantages: the file cannot be completed if there are **no seeders**, other users can see your **IP address**, and files may contain **malware** or be shared illegally.

Conclusion: torrent divides a file into pieces and shares them among many peers at the same time, using a tracker or DHT to find peers and hashes to check each piece. This makes sharing large files fast and scalable.

5. Advantages of Peer-to-Peer over Client-Server Architecture

In the client-server architecture, client processes interact with server processes on different computers to access the shared resources that the servers manage. In the peer-to-peer (P2P) architecture, all processes involved in a task have similar roles. They interact cooperatively as peers, with no distinction between client and server processes. Based on this difference, P2P has the following advantages over client-server:

1. Resources grow with the number of users. In P2P, the network and computing resources owned by each user can also be used to support the service. So the resources available to run the service grow with the number of users. In client-server, all shared resources are managed by the server, so more clients only add more load to the server.
2. Uses the resources of all participating computers. The aim of P2P is to exploit the resources of the participating computers to complete a task. P2P systems let millions of computers give access to data and resources that they store and manage together, instead of depending on one server's resources.
3. Better load distribution. Each computer holds only a small part of the application database, storage, processing and communication load. In client-server, the load falls on the server or servers.
4. Resilience in case of disconnection. Each data object is replicated on several computers. This spreads the load further and keeps the system working if any individual computer disconnects.

In client-server, clients can't access the resources if the server that manages them is unavailable.
5. Equal roles for all peers. All participating processes run the same program and offer the same set of interfaces to each other. Any peer can both request and provide a service, so the system doesn't rely on special dedicated machines.
6. Cost-effective. One goal of an architectural model is to make the system reliable, manageable, adaptable and cost-effective. P2P helps with this by reusing the users' own resources instead of adding more servers, which is how client-server has to scale (by mapping services to multiple servers).

6. Between Client-Server and Peer-to-Peer, Which One Is More Complex?
Peer-to-peer is more complex than client-server.
Client-server is simpler because:
• Roles are clearly separated. Client processes send requests, and server processes manage the shared resources and reply.
• Communication follows the request-reply protocol, which is a simple pairwise exchange of messages from client to server and from the server back to the client.
• It's supported by remote procedure calls (RPC). The server offers its operations through a service interface, and clients call them as if they were local. RPC hides the encoding and decoding of parameters and results and the passing of messages.• Shared resources are managed in one place, by the server.
Peer-to-peer is more complex because:
• No fixed roles: All processes play similar roles. Each peer must act as both client and server, running the same program and offering the same interfaces to the others.
• Communication pattern: Applications are made of a large number of peer processes running on all computers, and the communication pattern changes with the application's requirements. It is not a simple request-reply pattern.
• Distributed data: A large number of data objects are shared, and each computer holds only a small part of the database, storage and processing. The system has to handle the partitioning, placement and retrieval of shared data objects among the cooperating computers.
• Replication: Each object is replicated on several computers and has to keep working when any computer disconnects. Managing these copies adds complexity.
• Special middleware: P2P needs dedicated middleware, such as routing overlays (Pastry, Tapestry) and application-specific systems (Gnutella, OceanStore, Ivy, Squirrel), to locate and route to data and peers.

7. Are multiple servers and a cluster the same?
No, they are not the same, although a cluster is built from multiple servers. Multiple servers just means you have more than one server. Each one may run independently, serve a different purpose, and know nothing about the others. For example, one machine hosts your website, another hosts

your email, and a third runs your database. If one goes down, the others carry on, but nothing takes over its work automatically. A cluster is a group of servers that are connected and managed so they behave like a single system. They share a common goal, such as handling the same workload. A cluster usually has these traits.

- Coordination through cluster management software, with the nodes aware of each other (often via heartbeat signals)
- Load balancing, so requests are spread across nodes
- Failover, so if one node fails another takes over its work automatically
- A shared view of data or state, so users see one consistent service
- Easy scaling by adding more nodes

In short, multiple servers are a collection, while a cluster is a collection that is organized to work together as one. Common cluster types are high-availability clusters, load-balancing clusters, and high-performance computing clusters.

8. Explain proxy server.

A proxy server is an intermediary that sits between a client (such as your browser) and the destination server (such as a website). Instead of connecting directly, the client sends its request to the proxy, the proxy forwards it to the destination, then returns the response to the client.

How it works in steps.

1. The client sends a request to the proxy.
2. The proxy may check rules, look in its cache, or modify the request.
3. The proxy forwards the request to the target server using its own IP address.
4. The target server replies to the proxy.
5. The proxy passes the reply back to the client.

There are two main types.

- **Forward proxy** sits in front of clients. It hides client identities, filters content, and caches pages. Schools and companies use it to control internet access.
- **Reverse proxy** sits in front of servers. It hides the real servers, balances load, handles SSL/TLS encryption, and caches content. Nginx and HAProxy are common examples.